

Advanced Natural Language Processing

Assignment 4: Transformer Architecture

Lucia Welther, Matrikelnummer: 835106

January 21, 2026

1 Architecture

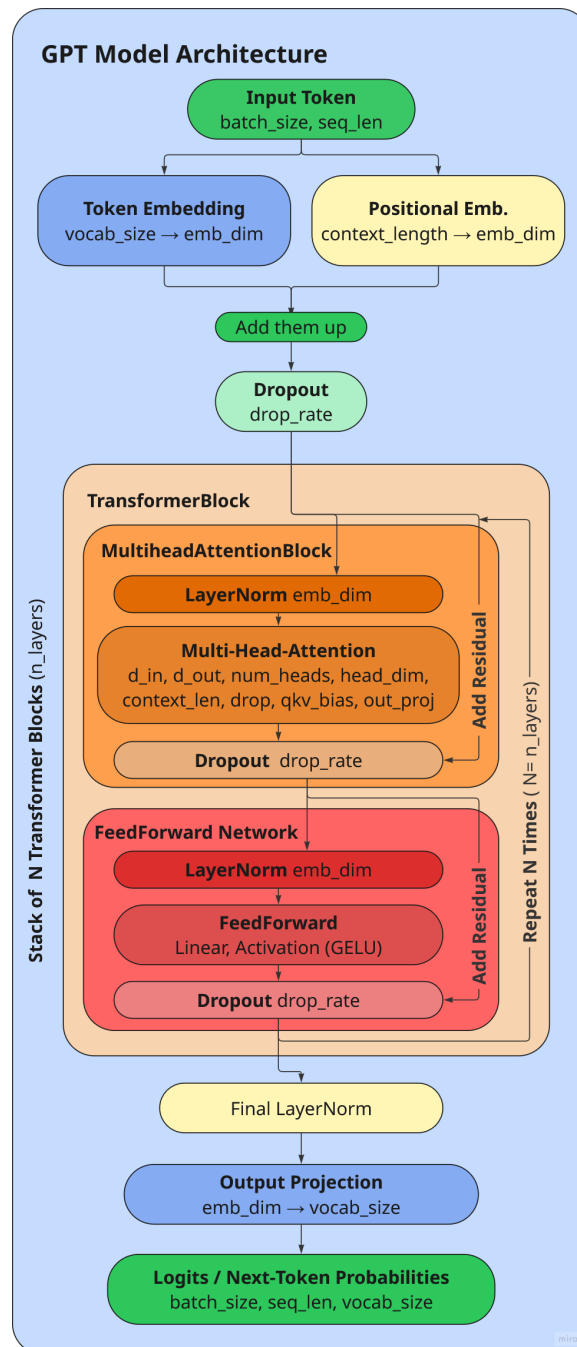


Figure 1: GPT Model Architecture

1.1 Description of Architecture

In Figure 1 the GPT model is shown. A sequence of token IDs is mapped to dense vectors by a token embedding ($\text{vocab_size} \rightarrow \text{emb_dim}$) and a positional embedding ($\text{context_length} \rightarrow \text{emb_dim}$); the two embeddings are summed and regularized with dropout. The resulting tensor of shape $(\text{batch}, \text{seq_len}, \text{emb_dim})$ is processed by a stack of N identical TransformerBlocks.

Each TransformerBlock performs:

1. Pre-LayerNorm on the input.
2. Multi-head causal self-attention with n_heads (head dimension = $\text{emb_dim} / n_heads$). The attention output is projected, dropped out, and added to the residual connection.
3. Pre-LayerNorm on the output from Multi-head attention Block
4. Feed-forward network with GELU activation: $\text{emb_dim} \rightarrow 4 \cdot \text{emb_dim} \rightarrow \text{GELU} \rightarrow \text{emb_dim}$, then dropout and a residual add.

After all layers have been passed through, a final LayerNorm and a linear projection create logits/probabilities over the vocabulary for next-token prediction.

2 Evaluate the Model

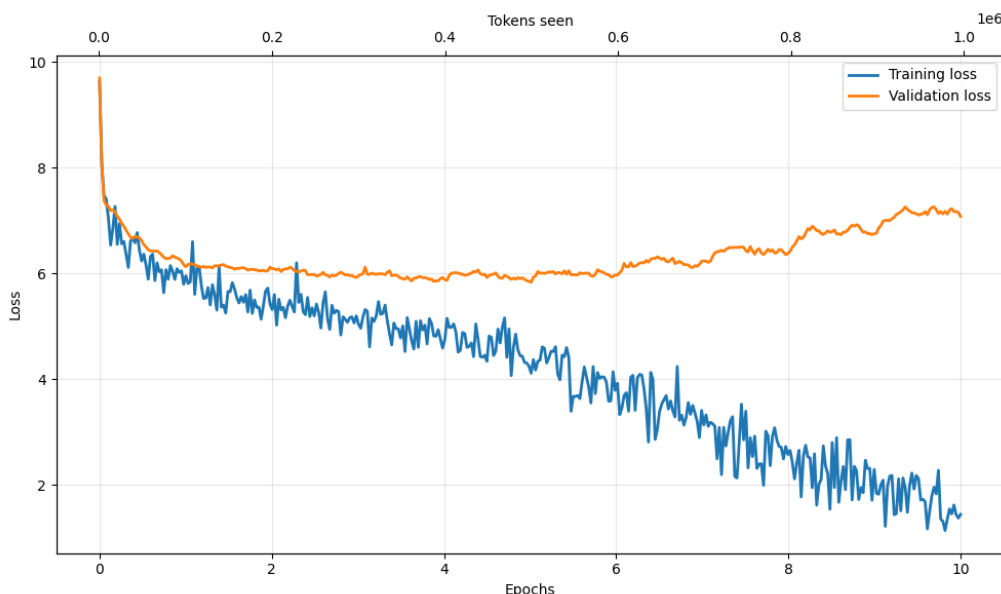


Figure 2: GPT Model Loss Plot

In the Figure 2 you can see how the training loss is constantly dropping over the 10 epochs, while the validation loss is first dropping as fast as the training loss, then in a slow pace until epoch 4, until the loss increases again after. This shows that the model is overfitting to the training data, performing well and always improving on the training data but it does not improve in generalizing on the unseen validation data.

During training the output created changed during the epochs. In the first 4 epochs the model generated only one or two new words: “My beloved Sister, and the”. Only after longer sequences are generated. In Epoch 5, 6 and 7 the model got stuck in repeating word sequences and repeating white spaces: “... the most the most beautiful and the most the most the most the ...” (Epoch 5). With each epoch the repeating word sequences got longer. In Epoch 8 the model stopped repeating the same sequences. These generated sequence of words, did not sound natural nor was grammatical. For the last episodes the sequences read more naturally but still do not have any semantic value.

2.1 Perplexity

Table 1 presents the perplexity results for the two test corpora (Frankenstein, Automata). The model has a perplexity of 207.8 on Frankenstein, which can be translated into high uncertainty. The die_automata corpus reached even worse results. The perplexity is extremely high (883,024.9), meaning that the model assigns probabilities close to zero to most tokens. This extreme difference is probably due to the mismatch in language (German vs. English). With the model being trained on English text, it generalized very badly on German text (Die Automate). Training the model on German Text would improve the perplexity on German test corpora.

Table 1: Perplexity evaluation on test corpora

Test Corpus	Tokens Evaluated	Avg. NLL	Perplexity
frankenstein_short.txt	599	5.34	207.80
die_automata.txt	12,470	13.69	883,024.94

3 Decoding

3.1 Implementation

Greedy Decoding (already implemented Baseline): The baseline `generate_text_simple()` always selects the token with maximum probability at each step. **Top-k Sampling:** Implementation inspired by: Ansil, M. B. (2025, January 5). The `decode_1` function keeps only the k most probable tokens, renormalizes, and samples: Sort logits descending, keep top k , Mask remaining logits to $-\infty$ and Apply softmax and sample from distribution. $k = 50$ and temperature $T = 1.0$ were used as default.

Nucleus (Top-p) Sampling: Implementation inspired by: Ansil, M. B. (2025, January 5). The `decode_2` function samples from the smallest set of tokens whose cumulative probability exceeds threshold p : Sort logits descending, Compute cumulative probabilities, Keep tokens while $\sum p_i \leq p$, Mask rest to $-\infty$ and sample. $p = 0.9$ was used as default.

3.2 Comparison

Greedy decoding output:

My beloved Sister, and "You will doubtless?" "If you, dear Margaret, that I am not, that you have been so much: you are not, that the monster?" "Devratitude. If you have been, that you,
(Gets stuck in repetition loop)

Top-k sampling output ($k = 50$):

My beloved Sister, but not a inspired me. "If you," said he hasably from thank you, what these friends with a pretty not be:." "Dear, that the stranger was so many years, and the very unlike of

Nucleus sampling output ($p = 0.9$):

My beloved Sister, I gazed and impassle with you, and said, hated on you; you alone my heart." "This is you how more "EL-"Dev need you in my; you will befitting kindness;

- **Greedy:** Deterministic but prone to repetition loops
- **Top-k:** Introduces diversity; fixed k may include unlikely tokens or exclude likely ones
- **Top-p:** Dynamically adjusts candidate pool based on probability mass, often producing more natural variation

4 Hyperparameter Experiments

GPT-2 Small (124M)

With parameters: “emb_dim”: 768, “n_layers”: 12, “n_heads”: 12, “epochs”: 10

My beloved Sister, and "You will doubtless?" "If you, dear Margaret, that I am not, that you have been so much: you are not, that the monster?" "Devratitude. If you have been, that you,

GPT-2 Medium (355M)

With parameters: “emb_dim”: 1024, “n_layers”: 24, “n_heads”: 16, “epochs”: 8

My beloved Sister, and "I have not to the s of the s of the s of the s of the s of the the monster and s of the s of the s of the s of

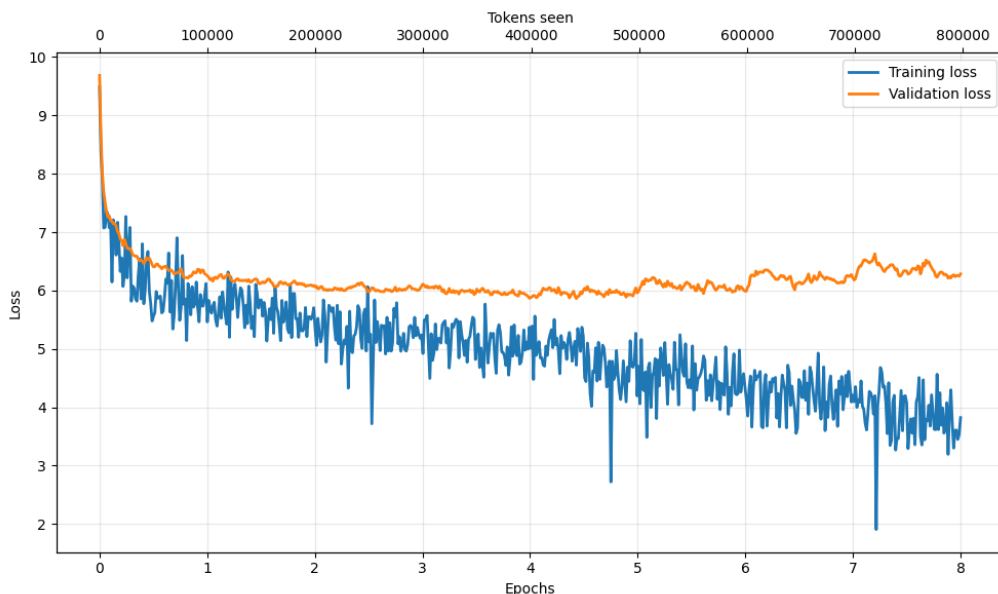


Figure 3: GPT Model Loss Plot: gpt2 medium (355M)

I used fewer epochs for the medium model to save time and reduce complexity. The performance of the medium model was comparable to the small model after 4 iterations, but both models did not show optimal performance. The medium model also struggled with generating coherent sequences and got stuck in a sequence repetition loop (like the small model in earlier epochs). Interestingly, up until the last epoch, the medium model frequently produced only one word after the starting context.

The medium model, with its larger number of parameters, requires more training data and epochs to fully converge. But time constraint made it impossible to train.

In the loss plot for the medium model, a similar behavior to the small model can be observed. The training loss decreases steadily, while the validation loss remains relatively constant and even increases slightly towards the end, indicating overfitting.

The larger model parameters, did not run on my computers cpu, so I do not have results for this. I would guess that the performance would not improve with too many and big parameters. To improve the model I would improve the training data to be longer and more complex in vocabulary and language for the model to learn more complex language patterns.

5 Pretrained vs. From-Scratch Comparison

My beloved Sister, I am so sorry for your loss. I am so sorry for your loss. I am so sorry for your loss. I am so sorry for your loss. I am so sorry for your loss. I am so sorry for your loss. I am so sorry for your loss.

your loss. I am

The pretrained GPT-2 model generates grammatically correct sentences that are coherent and fluent but the output gets stuck in repeating patterns, as seen in the repeated phrase "I am so sorry for your loss." This shows that while the pretrained model has strong language fluency, it may rely too heavily on high-probability tokens, leading to repetitive outputs.

In comparison, the self-trained model produces outputs with less grammatical correctness and fluency but the choice of words and sentence structure occasionally aligns better with the training data's style. Looking at earlier epochs in training of the self-trained model repetition can also be seen.

References

Ansil, M. B. (2025, January 5). *From randomness to precision: How top-k, top-p, temperature, and beam search shape text generation*. Medium. Retrieved from <https://medium.com/@ansilproabl/from-randomness-to-precision-how-top-k-top-p-temperature-and-beam-search-shape-text-generation-d1f50b5220e2>